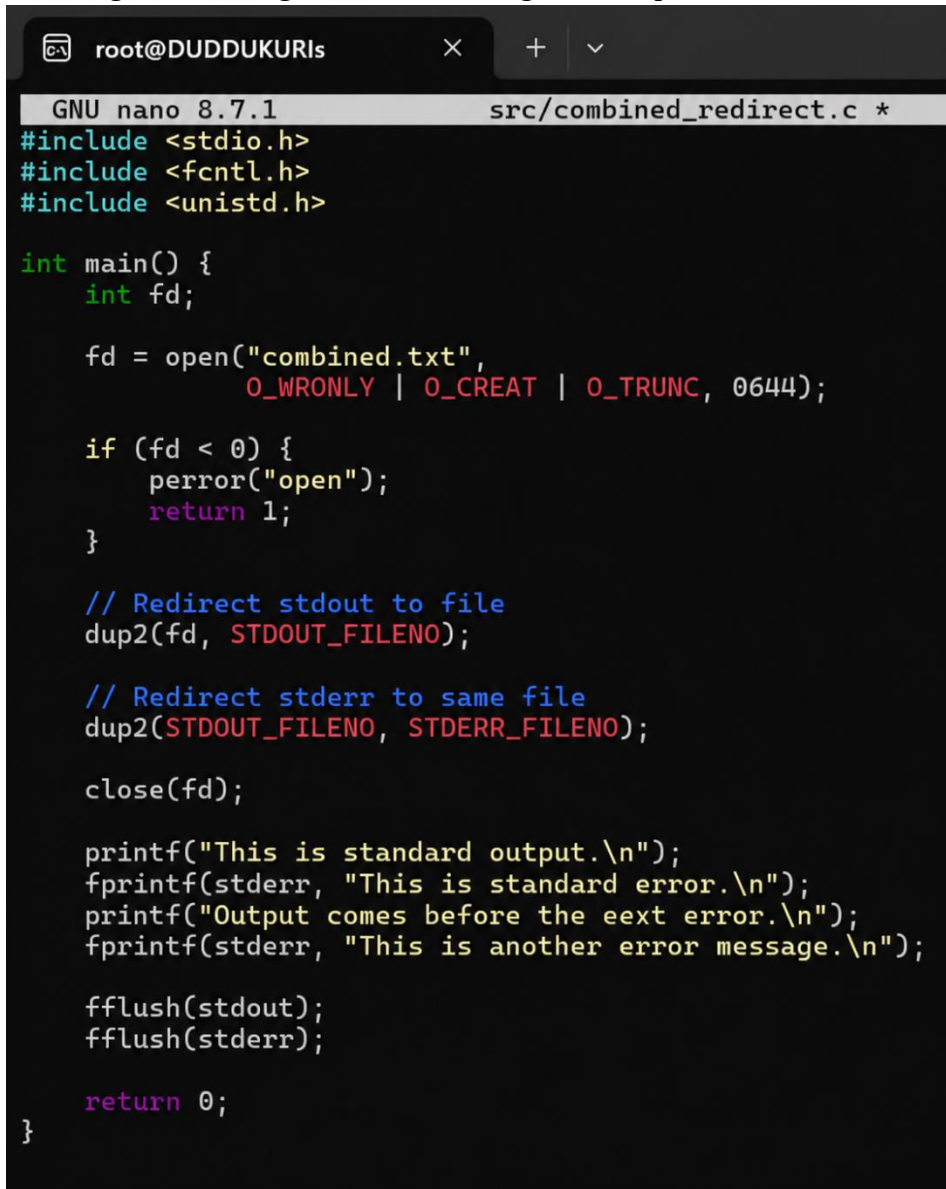


Week 8

1.To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs



```
GNU nano 8.7.1 src/combined_redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    fd = open("combined.txt",
              O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);

    // Redirect stderr to same file
    dup2(STDOUT_FILENO, STDERR_FILENO);

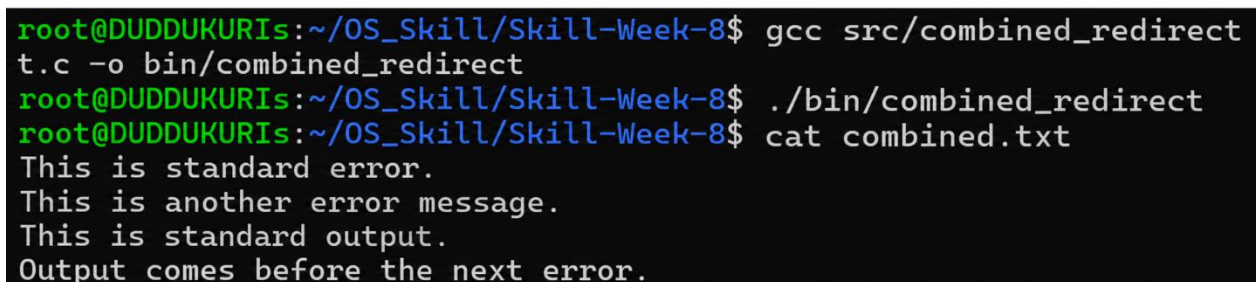
    close(fd);

    printf("This is standard output.\n");
    fprintf(stderr, "This is standard error.\n");
    printf("Output comes before the next error.\n");
    fprintf(stderr, "This is another error message.\n");

    fflush(stdout);
    fflush(stderr);

    return 0;
}
```

Output:



```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ gcc src/combined_redirect
t.c -o bin/combined_redirect
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/combined_redirect
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ cat combined.txt
This is standard error.
This is another error message.
This is standard output.
Output comes before the next error.
```

2.To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
root@DUDDUKURIs × + ▾
GNU nano 8.7.1 src/command_parser.c *
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char copy[200];
    char *cmd;
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Invalid command.\n");
        return 1;
    }

    strcpy(copy, input);

    printf("\nExecution Plan:\n");

    cmd = strtok(copy, "|");

    while (cmd != NULL) {
        count++;
        printf("Command %d: %s\n", count, cmd);
        cmd = strtok(NULL, "|");
    }

    if (strstr(input, ">") != NULL)
        printf("Output redirection detected.\n");

    if (strstr(input, "<") != NULL)
        printf("Input redirection detected.\n");

    if (strstr(input, ">>") != NULL)
        printf("Input redirection detected.\n");

    if (strchr(input, '|') != NULL)
        printf("Pipe detected.\n");

    printf("Total commands: %d\n", count);
    printf("Execution order: Left to Right.\n");

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ gcc src/command_parser.c -o bin/
command_parser
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/command_parser
Enter command: ls | grep .c > result.txt

Execution Plan:
Command 1: ls
Command 2: grep .c > result.txt
Output redirection detected.
Pipe detected.
Total commands: 2
Execution order: Left to Right.
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/command_parser
Enter command: ls | grep .c

Execution Plan:
Command 1: ls
Command 2: grep .c
Pipe detected.
Total commands: 2
Execution order: Left to Right.
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/command_parser
Enter command: cat < input.txt | grep Hello

Execution Plan:
Command 1: cat < input.txt
Command 2: grep Hello
Input redirection detected.
Pipe detected.
Total commands: 2
Execution order: Left to Right.
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/command_parser
Enter command: ls 2> errors.txt

Execution Plan:
Command 1: ls 2> errors.txt
Output redirection detected.
Error redirection detected.
Total commands: 1
Execution order: Left to Right.
```

3. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
root@DUDDUKURIs × + v
GNU nano 8.7.1 src/background_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    printf("myshell> ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    int len = strlen(command);

    // Check for background operator &
    if (len > 0 && command[len - 1] == '&') {
        command[len - 1] = '\0';

        pid = fork();

        if (pid < 0) {
            perror("fork");
            return 1;
        }

        if (pid == 0) {
            printf("Background job started. PID: %d\n", getpid());

            sleep(5);

            printf("Background job completed. PID: %d\n", getpid());
            exit(0);
        }

        printf("[1] Background job PID: %d\n", pid);
        printf("myshell> Prompt returned immediately.\n");

        sleep(1);

        if (waitpid(pid, NULL, WNOHANG) == 0)
            printf("Job is still running.\n");

        sleep(5);

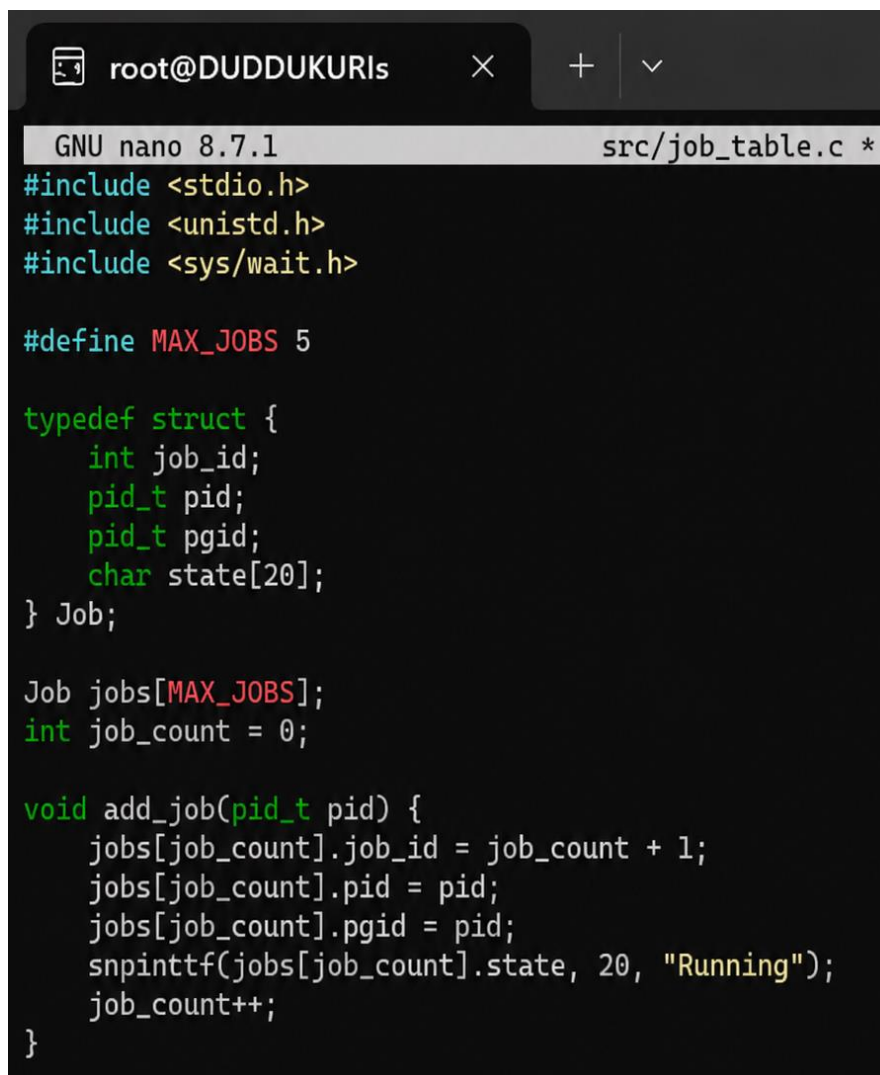
        waitpid(pid, NULL, 0);
        printf("Job completed.\n");
    }
    else {
        printf("Foreground command detected.\n");
    }

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ gcc src/background_jobs.c -o bin/
/background_jobs
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/background_jobs
mysHELL> sleep 5 &
[1] Background job started. PID: 1045
mysHELL> sleep 2
mysHELL> Background job completed. PID: 1045
[1] Background job PID: 1045
mysHELL>
```

4. To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records



```
root@DUDDUKURIs  ×  +  ▾
GNU nano 8.7.1 src/job_table.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 5

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pid;
    snprintf(jobs[job_count].state, 20, "Running");
    job_count++;
}
```

```

void show_jobs() {
    printf("\nJob Table:\n");

    for (int i = 0; i < job_count; i++) {
        printf("[%d] PID=%d PGID=%d State=%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].state);
    }
}

int main() {
    pid_t pid;

```

```

    pid = fork();

    if (pid == 0) {
        setpgid(0, 0);
        sleep(3);
        return 0;
    }

    add_job(pid);

    show_jobs();

    sleep(1);

    if (waitpid(pid, NULL, WNOHANG) == 0)
        printh("\nJob is still running.\n");

    waitpid(pid, NULL, 0);

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].pid == pid) {
            snprintf(jobs[i].state, 20, "Completed");
        }
    }

    show_jobs();

    // Remove completed job
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].pid == pid) {
            for (int j = i; j < job_count - 1; j++)
                jobs[j] = jobs[j + 1];

            job_count--;
            break;
        }
    }

    printf("\nAfter removing completed job:\n");
    show_jobs();

    return 0;
}

```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ nano src/job_table.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ gcc src/job_table.c -o bin/job_table
root@DUDDUKURIs:~/OS_Skill/Skill-Week-8$ ./bin/job_table

Job Table:
[1] PID=1423 PGID=1423 State=Running

Job is still running.

Job Table:
[1] PID=1423 PGID=1423 State=Completed

After removing completed job:

Job Table:
```